Document number	P0405R0
Date	2015-06-28
Project	Programming Language C++, Library Working Group
Reply-to	Jonathan Wakely <cxx@kayari.org>

Wording for Networking TS changes discussed in Kona

Introduction

In Kona LEWG agreed a few changes to the Networking TS. Due to the C++17 schedule those changes were not yet reviewed by LWG.

This paper provides wording for those changes, as well as some definitions accidentally omitted from the original proposal that formed the first TS working draft.

All changes are relative to N4588. The proposed wording changes are not all presented at the end, but interleaved throughout the document. The project editor for the TS is happy with this presentation.

Removing default template argument of `async_result`

The SFINAE checking made possible by the final template parameter of `async_result` was deemed to be unnecessary. LEWG recommended its removal.

LEWG Discussion Notes

> Issue 224: Consider SFINAE-enabling associators
>
> Of the three main customisation points `async_result`, `associated_executor` and `associated_allocator`, only `async_result` has a `class = void` template parameter. Some LWG members strongly disliked this, and in the interest of consistency it may be best if either `async_result` loses it or the other two gain it.

Proposed Wording

Modify the header synopsis in 13.1 [async.synop]:

```
template<class CompletionToken, class Signature~~, class = void~~>
  class async_result;
```

Modify the class synopsis in 13.3 [async.async.result]:

```
template<class CompletionToken, class Signature~~, class = void~~>
class async_result
{
```

Add `operator+=` to buffer types

Based on implementation experience, the lack of a `+=` operator is a bothersome inconvenience when working with buffers. LEWG agreed such an addition would be useful.

LEWG Discussion Notes

> Issue 225: Consider operator+= for const_buffer and mutable_buffer
>
> The buffer classes already have `operator+`.
>
> Marshall implemented `operator+=` in his own implementation. Jonathan Wakely and Chris Kohlhoff both (independently) wished it was there when testing out Jon's implementation.
>
> One LWG member didn't like that +/+= were saturating.
>
> `string_view` uses the name `remove_prefix` for a similar operation.

Proposed Wording

Add to the class synopsis in 16.4 [buffer.mutable]:

```
class mutable_buffer
{
public:
  // constructors:
  mutable_buffer() noexcept;
  mutable_buffer(void* p, size_t n) noexcept;
  // members:
  void* data() const noexcept;
  size_t size() const noexcept;
  mutable_buffer& operator+=(size_t n) noexcept;
```

Add the function definition to the end of 16.4 [buffer.mutable]:

> `mutable_buffer& operator+=(size_t n) noexcept;`
>
> -6- *Effects:* Sets `data_` to `static_cast<char*>(data_) + min(n, size_)`, and then `size_` to `size_ - min(n, size_)`.
>
> -7- *Returns:* `*this`.

Add to the class synopsis in 16.5 [buffer.const]:

```
class const_buffer
{
public:
  // constructors:
  const_buffer() noexcept;
  const_buffer(const void* p, size_t n) noexcept;
  const_buffer(const mutable_buffer& b) noexcept;
  // members:
  const void* data() const noexcept;
  size_t size() const noexcept;
  const_buffer& operator+=(size_t n) noexcept;
```

Add the function definition to the end of 16.5 [buffer.const]:

> `const_buffer& operator+=(size_t n) noexcept;`
>
> -7- *Effects:* Sets `data_` to `static_cast<const char*>(data_) + min(n, size_)`, and then `size_` to `size_ - min(n, size_)`.
>
> -8- *Returns:* `*this`.

Shallow constness of executors.

During implementation a problem was discovered in the TS working paper, where comparison of const-qualified executor objects was specified in terms of calls to non-const member functions. The proposed resolution adds `const` to the accessors of executors. The new design is consistent with other handle types which do not propagate const in accessors, e.g. in the `operator*` and `get()` member functions of iterators and smart pointers.

LEWG Discussion Notes

> Issue 201: [io_context.exec.comparisons] requires calls to non-const functions
>
> Executor objects are shallow and copyable, so it may make sense for the `context()` member to be const-qualified. It's a similar relationship to that between `polymorphic_allocator` and `memory_resource`.

Proposed Wording

Modify 13.2.2 [async.reqmts.executor] p6 as shown, and modify Table 4, Executor requirements, to replace all seven occurrences of `cx1` with `x1` and all four occurrences of `cx2` with `x2`:

> In Table 4, `x1` and `x2` denote (possibly const) values of type `X`, ~~`cx1` and `cx2` denote (possibly const) values of type `X`,~~ `mx1` denotes an xvalue of type `X`, `f` denotes a `MoveConstructible` (C++Std [moveconstructible]) function object callable with zero arguments, `a` denotes a (possibly const) value of type `A` meeting the `Allocator` requirements (C++Std [allocator.requirements]), and `u` denotes an identifier.

Drafting note: The following edits simply add `const` to the accessors of the three classes `system_executor`, `executor`, and `strand`.

Modify the class synopsis in 13.18 [async.system.exec] to add `const` to the executor operations member functions:

```
  // executor operations:

  system_context& context() const noexcept;

  void on_work_started() const noexcept {}
  void on_work_finished() const noexcept {}

  template<class Func, class ProtoAllocator>
    void dispatch(Func&& f, const ProtoAllocator& a) const;
  template<class Func, class ProtoAllocator>
    void post(Func&& f, const ProtoAllocator& a) const;
  template<class Func, class ProtoAllocator>
    void defer(Func&& f, const ProtoAllocator& a) const;
};
```

Modify 13.18.1 [async.system.exec.ops] to add `const` to the member function signatures:

```
system_context& context() const noexcept;

[...]

template<class Func, class ProtoAllocator>
  void dispatch(Func&& f, const ProtoAllocator& a) const;

[...]

template<class Func, class ProtoAllocator>
  void post(Func&& f, const ProtoAllocator& a) const;
template<class Func, class ProtoAllocator>
  void defer(Func&& f, const ProtoAllocator& a) const;

[...]
```

Modify the class synopsis in 13.21 [async.executor] to add `const` to the executor operations member functions:

```
  // executor operations:

  execution_context& context() const noexcept;

  void on_work_started() const noexcept;
  void on_work_finished() const noexcept;

  template<class Func, class ProtoAllocator>
    void dispatch(Func&& f, const ProtoAllocator& a) const;
  template<class Func, class ProtoAllocator>
    void post(Func&& f, const ProtoAllocator& a) const;
  template<class Func, class ProtoAllocator>
    void defer(Func&& f, const ProtoAllocator& a) const;

  // executor capacity:
```

Modify 13.21.5 [async.executor.ops] to add `const` to the member function signatures:

```
execution_context& context() const noexcept;

[...]

void on_work_started() const noexcept;

[...]

void on_work_finished() const noexcept;

[...]

template<class Func, class ProtoAllocator>
  void dispatch(Func&& f, const ProtoAllocator& a) const;

[...]

template<class Func, class ProtoAllocator>
  void post(Func&& f, const ProtoAllocator& a) const;

[...]

template<class Func, class ProtoAllocator>
  void defer(Func&& f, const ProtoAllocator& a) const;

[...]
```

Modify the class synopsis in 13.25 [async.strand] to add `const` to the strand operations member functions:

```
  // strand operations:

  inner_executor_type get_inner_executor() const noexcept;

  bool running_in_this_thread() const noexcept;

  execution_context& context() const noexcept;

  void on_work_started() const noexcept;
  void on_work_finished() const noexcept;

  template<class Func, class ProtoAllocator>
    void dispatch(Func&& f, const ProtoAllocator& a) const;
  template<class Func, class ProtoAllocator>
    void post(Func&& f, const ProtoAllocator& a) const;
  template<class Func, class ProtoAllocator>
    void defer(Func&& f, const ProtoAllocator& a) const;
```

Modify 13.25.4 [async.strand.ops] to add `const` to the member function signatures:

```
execution_context& context() const noexcept;

[...]

void on_work_started() const noexcept;

[...]

void on_work_finished() const noexcept;

[...]

template<class Func, class ProtoAllocator>
  void dispatch(Func&& f, const ProtoAllocator& a) const;

[...]

template<class Func, class ProtoAllocator>
  void post(Func&& f, const ProtoAllocator& a) const;

[...]

template<class Func, class ProtoAllocator>
  void defer(Func&& f, const ProtoAllocator& a) const;

[...]
```

Remove unneeded `is_executor` specializations

This is a quasi-editorial change to remove some redundant specializations. It would be conforming for implementations to continue to provide the specializations as an optimization, but their existence has no normative effect.

Proposed Wording

Remove the template specialization from the synopsis in 13.21 [async.executor]:

```
    template<class Executor> const Executor* target() const noexcept;
  };

  template<> struct is_executor<executor> : true_type {};

  // executor comparisons:

  bool operator==(const executor& a, const executor& b) noexcept;
```

Remove the template specialization from the synopsis in 14.3 [io_context.exec]:

```
  bool operator!=(const io_context::executor_type& a,
                  const io_context::executor_type& b) noexcept;

  template<> struct is_executor<io_context::executor_type> : true_type {};

} // inline namespace v1
} // namespace net
} // namespace experimental
```

Define requirements for composed operations.

Mr. Kohlhoff writes:

> This important piece of specification was accidentally omitted when the original TR2 proposal was updated for the proposed technical specification. The wording in this pull request is intended to convey the intent and will certainly require wordsmithing.

Add a new sub-clause after 13.2.7.13 [async.reqmts.async.exceptions]

> **13.2.7.14 Composed asynchronous operations [async.reqmts.async.composed]**
>
> -1- In this Technical Specification, a *composed asynchronous operation* is an asynchronous operation that is implemented in terms of zero or more intermediate calls to other asynchronous operations. The intermediate asynchronous operations are performed sequentially. *[Note:* That is, the completion handler of an intermediate operation initiates the next operation in the sequence. *--end note]*
>
> An intermediate operation's completion handler shall have an associated executor that is either:
>
> - the type `Executor2` and object `ex2` obtained from the completion handler type `CompletionHandler` and object `completion_handler`; or
> - an object of an unspecified type satisfying the Executor requirements (13.2.2), that delegates executor operations to the type `Executor2` and object `ex2`.
>
> An intermediate operation's completion handler shall have an associated allocator that is either:
>
> - the type `Alloc2` and object `alloc2` obtained from the completion handler type `CompletionHandler` and object `completion_handler`; or
> - an object of an unspecified type satisfying the ProtoAllocator requirements (13.2.1 [async.reqmts.proto.allocator]), that delegates allocator operations to the type `Alloc2` and object `alloc2`.

Adjust 17.6 [buffer.async.read] p1 to use the new term, and add a reference to 13.2.7.14 [async.reqmts.async.composed]:

> -1- A composed asynchronous read operation (13.2.7.14, 16.2.4).

Adjust 17.8 [buffer.async.write] p1 to use the new term, and add a reference to 13.2.7.14 [async.reqmts.async.composed]:

> -1- A composed asynchronous write operation (13.2.7.14, 16.2.4).

Add a new paragraph to 17.10 [buffer.async.read.until] using the new term, with a reference to 13.2.7.14 [async.reqmts.async.composed]):

-?- A composed asynchronous operation (13.2.7.14).

-1- *Completion signature:* `void(error_code ec, size_t n)`.

Add new paragraphs to 20.2 [socket.algo.async.connect] using the new term, with references to 13.2.7.14 [async.reqmts.async.composed]):

-?- A composed asynchronous operation (13.2.7.14).

-1- *Completion signature:* `void(error_code ec, typename Protocol::endpoint ep)`.

[...]

-?- A composed asynchronous operation (13.2.7.14).

-7- *Completion signature:* `void(error_code ec, InputIterator i)`.

Acknowledgments

The wording changes in this proposal were all provided by Chris Kohlhoff via Git pull requests, all I have done is present them in this format for review. Many thanks to Chris for providing the changes.